

# Question Levels

## Scientific Machine Learning: Blending Science with Data

### Homework Day 12

- a) Download and plot the data from the file 'DataHomeworkDay12.csv'. You have two input variables 'x1data', 'x2data' and one output, 'ydata'. The data are randomly distributed.
- b) **(LEVEL 3)** Write a program in Julia that finds the best fitting Neural Network to the data in this file.

## Scientific Machine Learning: Blending Science with Data

### Homework Days 10+11

- a) Download and plot the data from the file 'DataHomeworkDay11.csv'
- b) **(LEVEL 1)** Write a program in Julia that finds the best fitting polynomial to the data in this file.
- c) **(LEVEL 2)** Try to fit the data to a Neural Network with 1 hidden layer and one node. You must use the `tanh()` activation function between in the input and the hidden layer. Can you find a good fit? Can you explain why?
- d) **(LEVEL 3)** Write a Julia program that uses a Neural Network to find a fit to this data set.

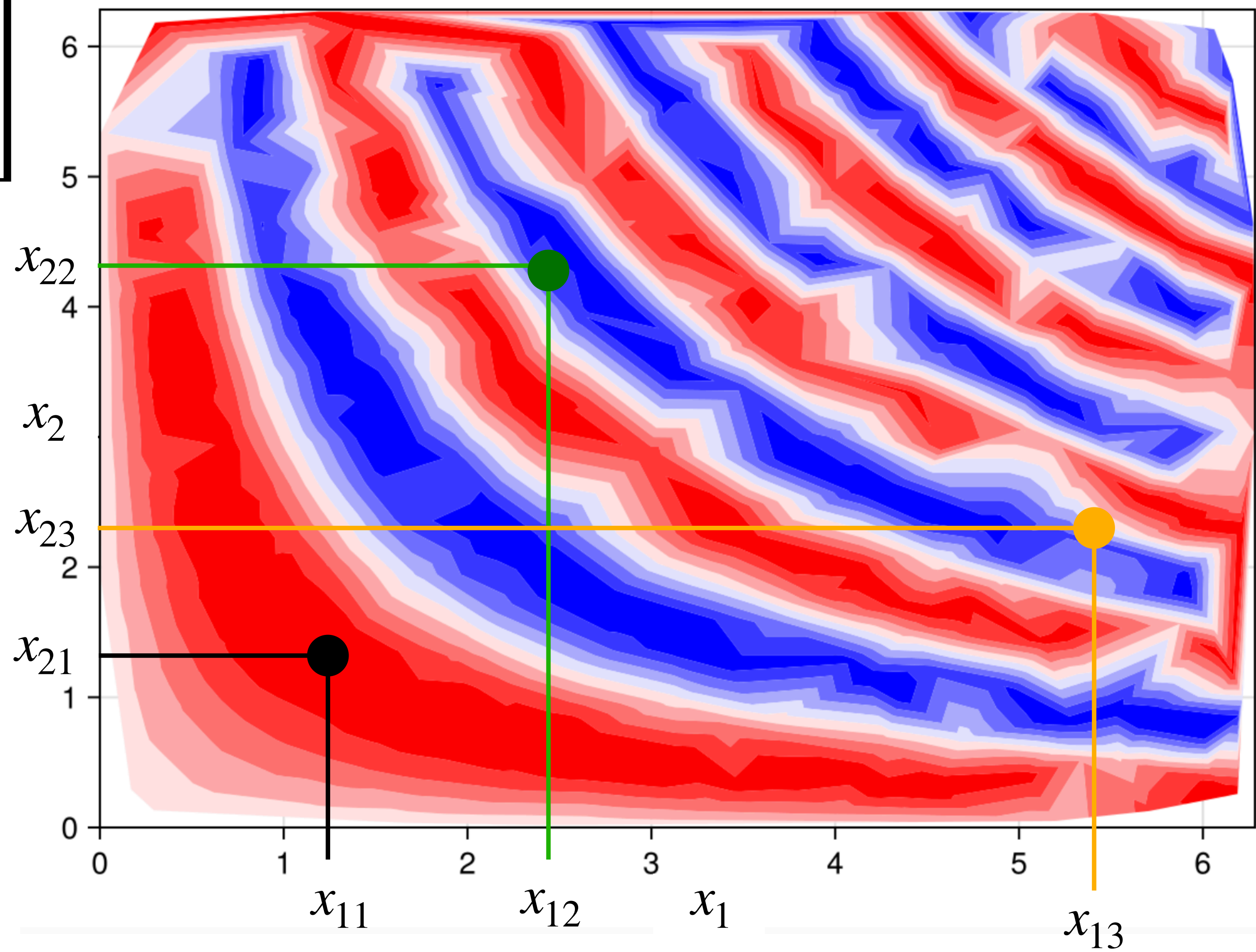
- Level 1 - No packages (except linear solvers) allowed.
- Level 2 - Automatic differentiation (e.g. ForwardDiff and Zygote) and linear solvers allowed.
- Level 3 - ALL Julia packages allowed.

# Exam (Discussion points/Not finalized)

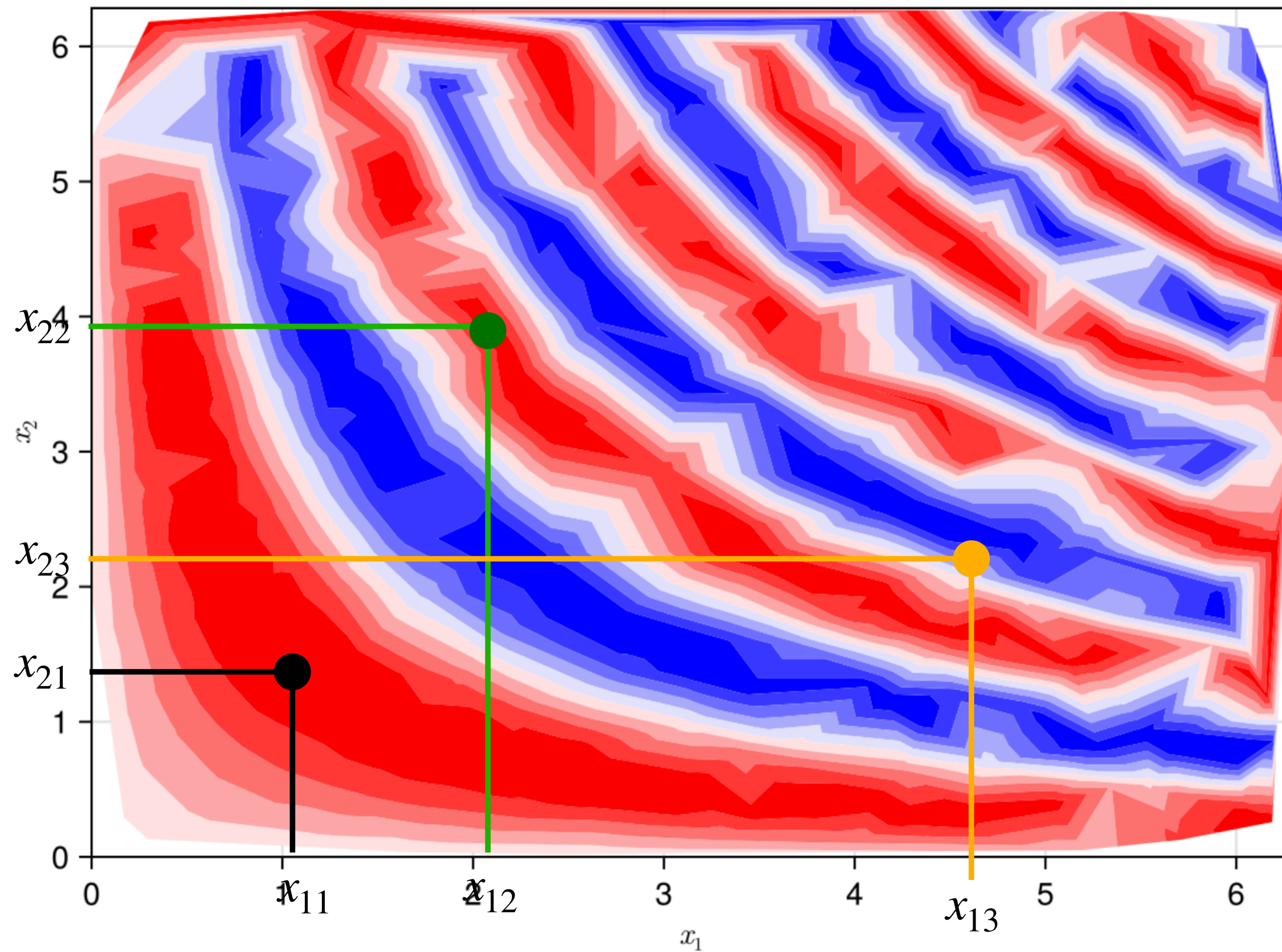
- NO internet or AI. Zhijun and myself will be supervising.
- Will give you 10 mins to download all data onto your computer.
- Will give questions in paper format after you have downloaded the data.
- Make sure you have Julia installed and download all packages that I have used in this course prior to the exam.
- Stop writing/typing 15 mins before the 3 hour mark for you to upload your files.
- 6 questions/Total time is 3 hours. Including upload/download time.
- Question Level need to be adhered to!
- Need to run “as is” from top to bottom. All packages must be “included” in the .jl/.ipynb files
- All questions must be answered and uploaded in separate .jl and .ipynb files. E.g. Question 1, is 1 file, Question 2 is one file etc.

$$\mathbf{X} = \begin{bmatrix} \boxed{x_{11}} & \boxed{x_{12}} & \boxed{x_{13}} & \cdots & x_{1N} \\ x_{21} & \boxed{x_{22}} & \boxed{x_{23}} & \cdots & x_{2N} \end{bmatrix}$$

$$\mathbf{y} = [\boxed{y_1}] \quad \boxed{y_2} \quad \boxed{y_3} \quad \cdots \quad y_N]$$







Let's try to model this data with the following model

$$f_{model}(x_1, x_2) = a_0 + a_1x_1 + a_2x_2 + a_3x_1x_2 + a_4x_1^2 + a_5x_2^2$$

Use normal equations method (or you can use standard gradient descent).

$$\mathbf{V} = \begin{bmatrix} 1 & x_{11} & x_{21} & x_{11}x_{21} & x_{11}^2 & x_{21}^2 \\ 1 & x_{12} & x_{22} & x_{12}x_{22} & x_{12}^2 & x_{22}^2 \\ 1 & x_{13} & x_{23} & x_{13}x_{23} & x_{13}^2 & x_{23}^2 \\ \vdots & & & & & \\ 1 & x_{1N} & x_{2N} & x_{1N}x_{2N} & x_{1N}^2 & x_{2N}^2 \end{bmatrix}$$

$$\vec{a} = (\mathbf{V}^T \mathbf{V})^{-1} \mathbf{V}^T \vec{y}$$

$$\mathbf{V} = \begin{bmatrix} 1 & x_{11} & x_{21} & x_{11}x_{21} & x_{11}^2 & x_{21}^2 \\ 1 & x_{12} & x_{22} & x_{12}x_{22} & x_{12}^2 & x_{22}^2 \\ 1 & x_{13} & x_{23} & x_{13}x_{23} & x_{13}^2 & x_{23}^2 \\ \vdots & & & & & \\ 1 & x_{1N} & x_{2N} & x_{1N}x_{2N} & x_{1N}^2 & x_{2N}^2 \end{bmatrix}$$

```
V=hcat(ones(N_SAMPLES),x1_samples[:,],x2_samples[:,],x1_samples[:,].*x2_samples[:,],x1_samples[:,].^2,x2_samples[:,].^2)
a=V'*V\'(V'*y_samples[:,])
```

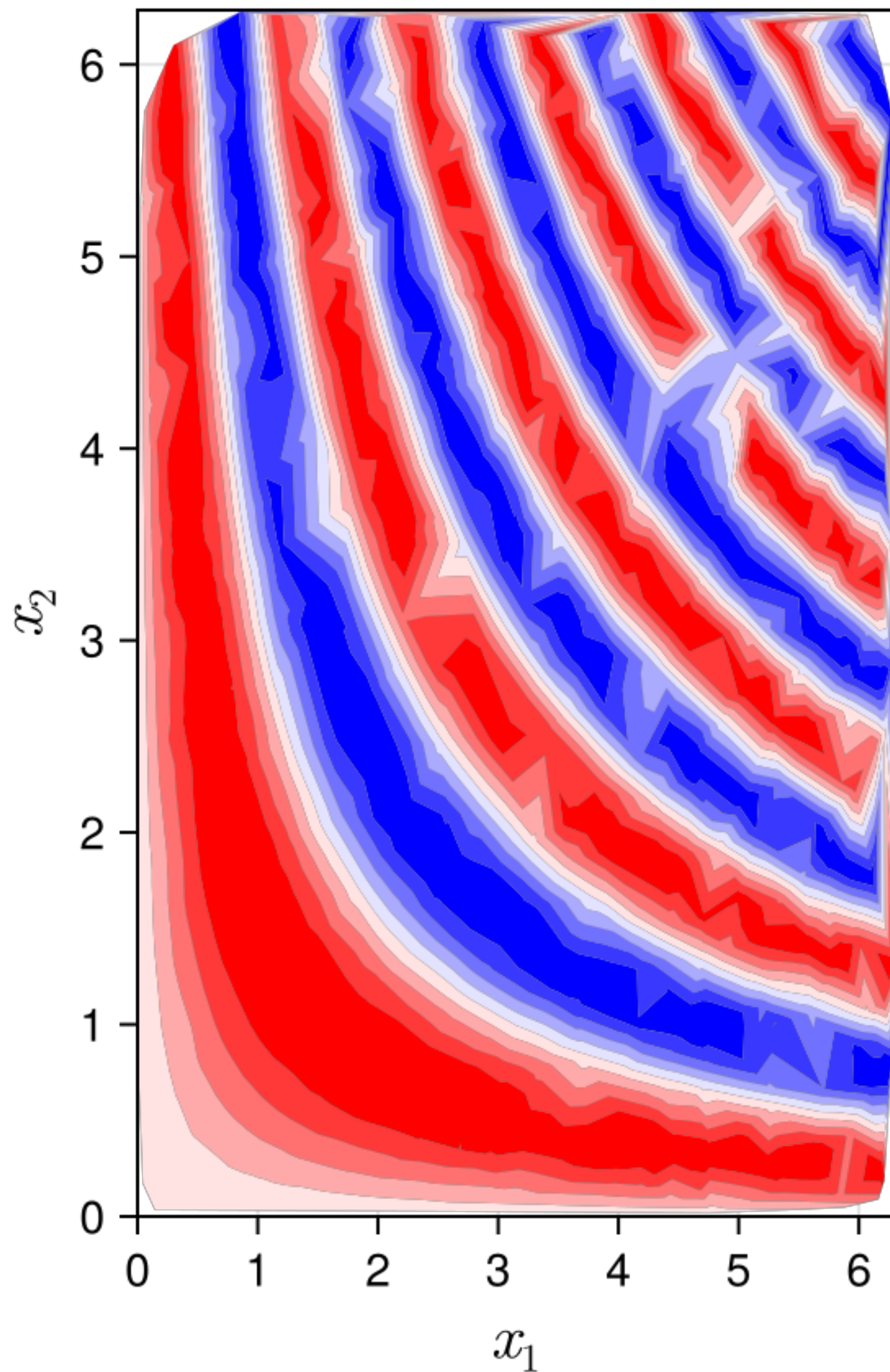
$$\vec{a} = (\mathbf{V}^T \mathbf{V})^{-1} \mathbf{V}^T \vec{y}$$

$$f_{model}(x_1, x_2) = a_0 + a_1 x_1 + a_2 x_2 + a_3 x_1 x_2 + a_4 x_1^2 + a_5 x_2^2$$

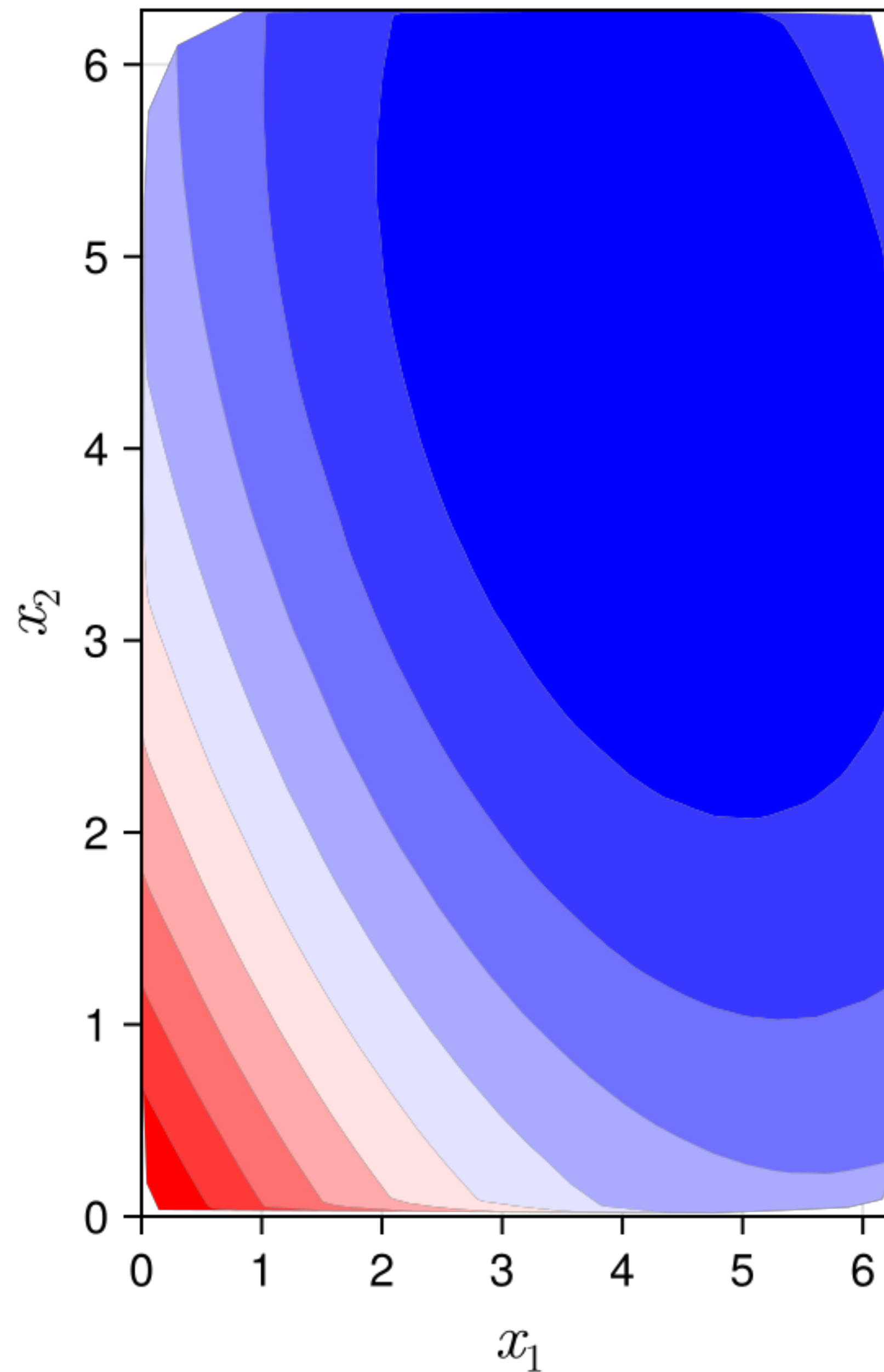
```
ypred=a[1].+a[2]*x1_samples[:,].+a[3]*x2_samples[:,].+a[4]*x1_samples[:,].*x2_samples[:,].+a[5]*x1_samples[:,].^2+a[6]*x2_samples[:,].^2
```



# Data



# Polynomial Model

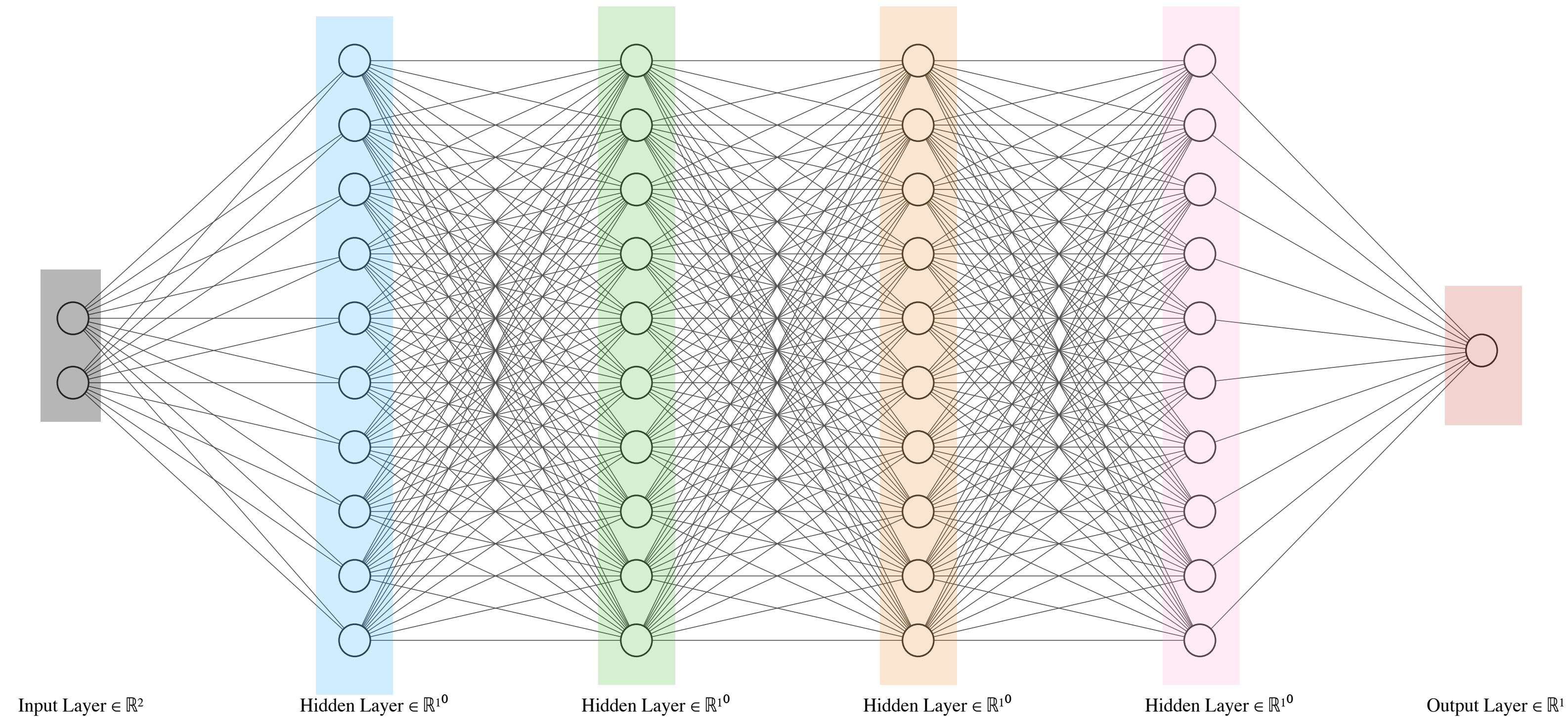


Agreement is  
not very good!  
Not a good model

$$f_{model}(x_1, x_2) = a_0 + a_1x_1 + a_2x_2 + a_3x_1x_2 + a_4x_1^2 + a_5x_2^2$$



Now let's train the Neural Network below to model the data

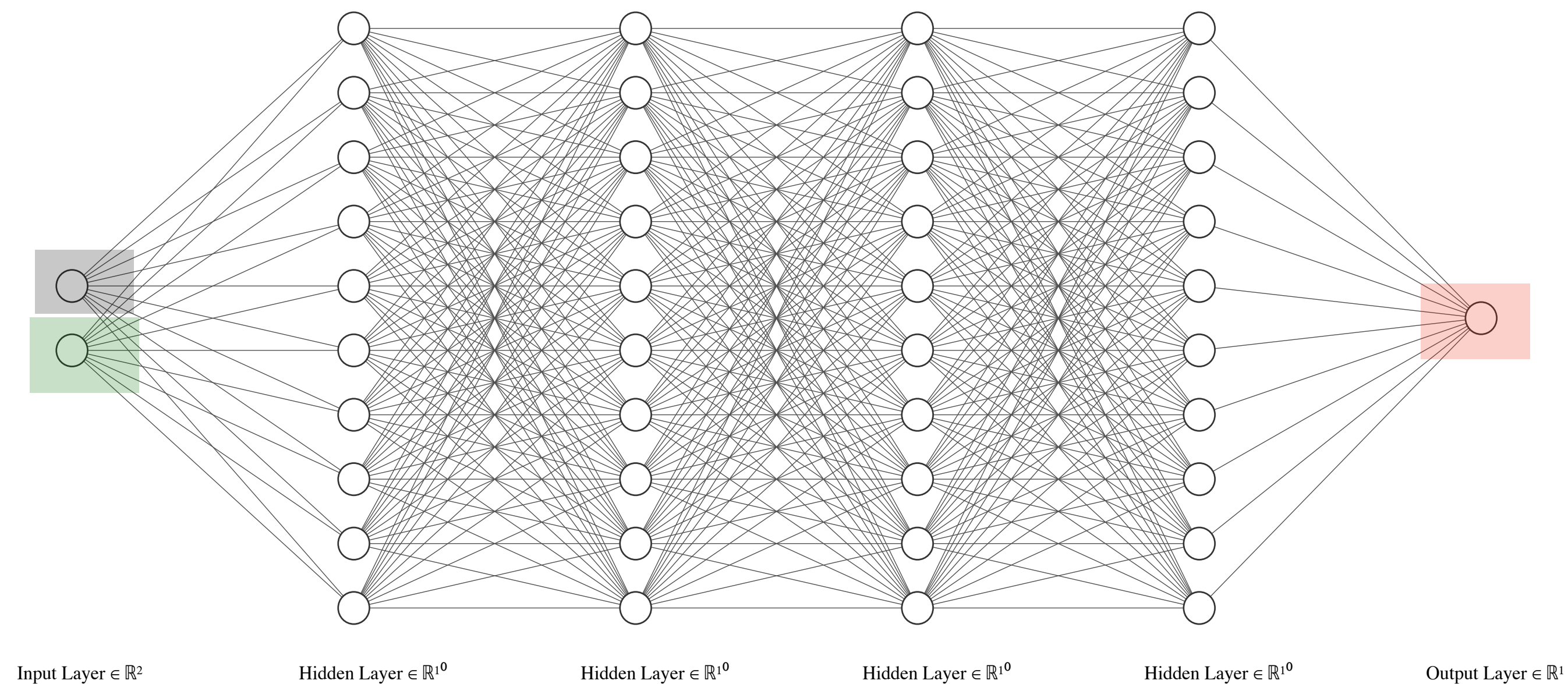


```
model = Chain(  
    Dense(2 => 10),  
    Dense(10 => 10, tanh),  
    Dense(10 => 10, tanh),  
    Dense(10 => 10, tanh),  
    Dense(10 => 1)  
)
```

```
x_samples=[Float32.(x1_samples);Float32.(x2_samples)]
```

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & \cdots & x_{1N} \\ x_{21} & x_{22} & x_{23} & \cdots & x_{2N} \end{bmatrix}$$

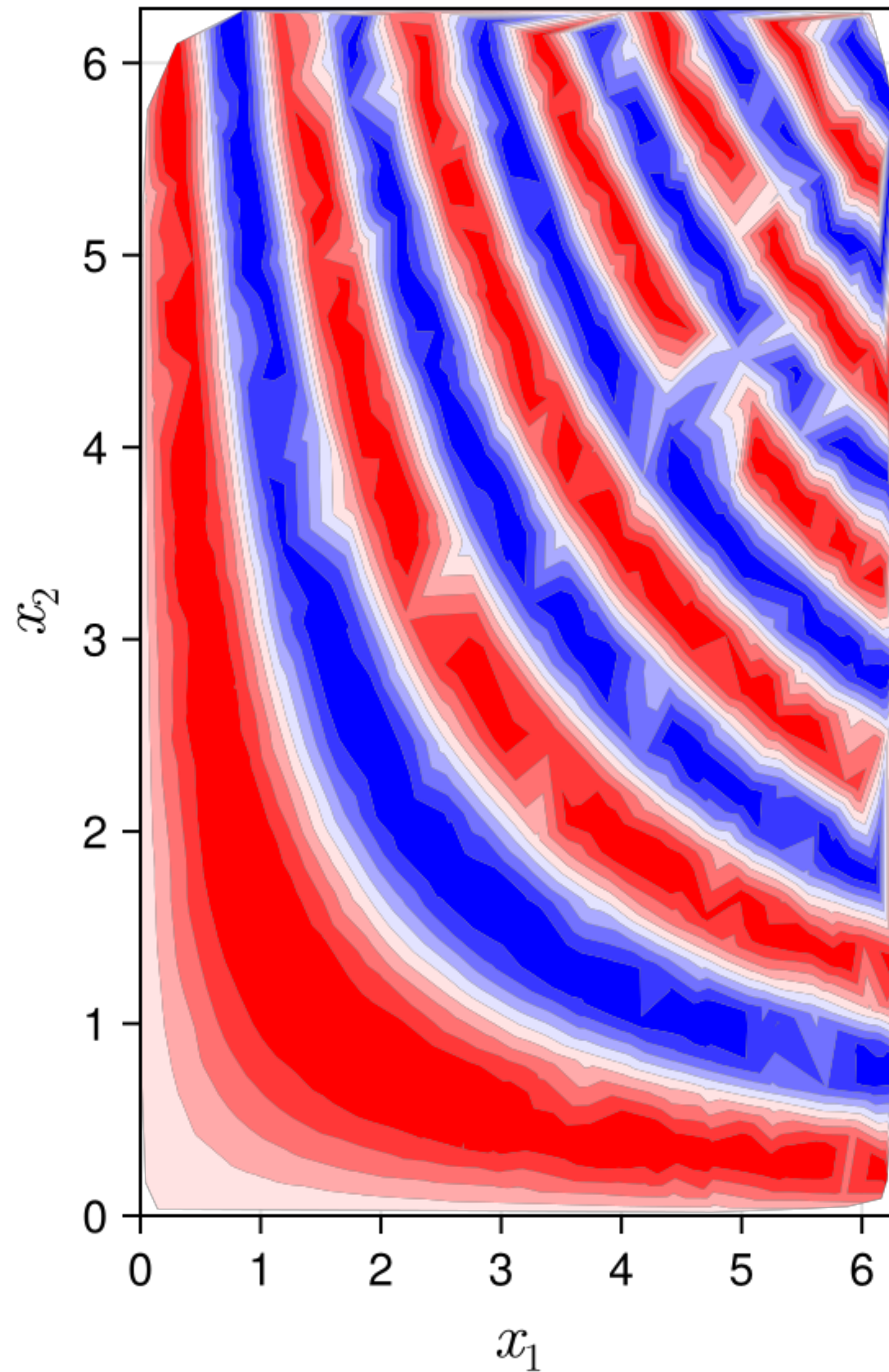
$$\mathbf{y} = [y_1 \ y_2 \ y_3 \ \cdots \ y_N]$$



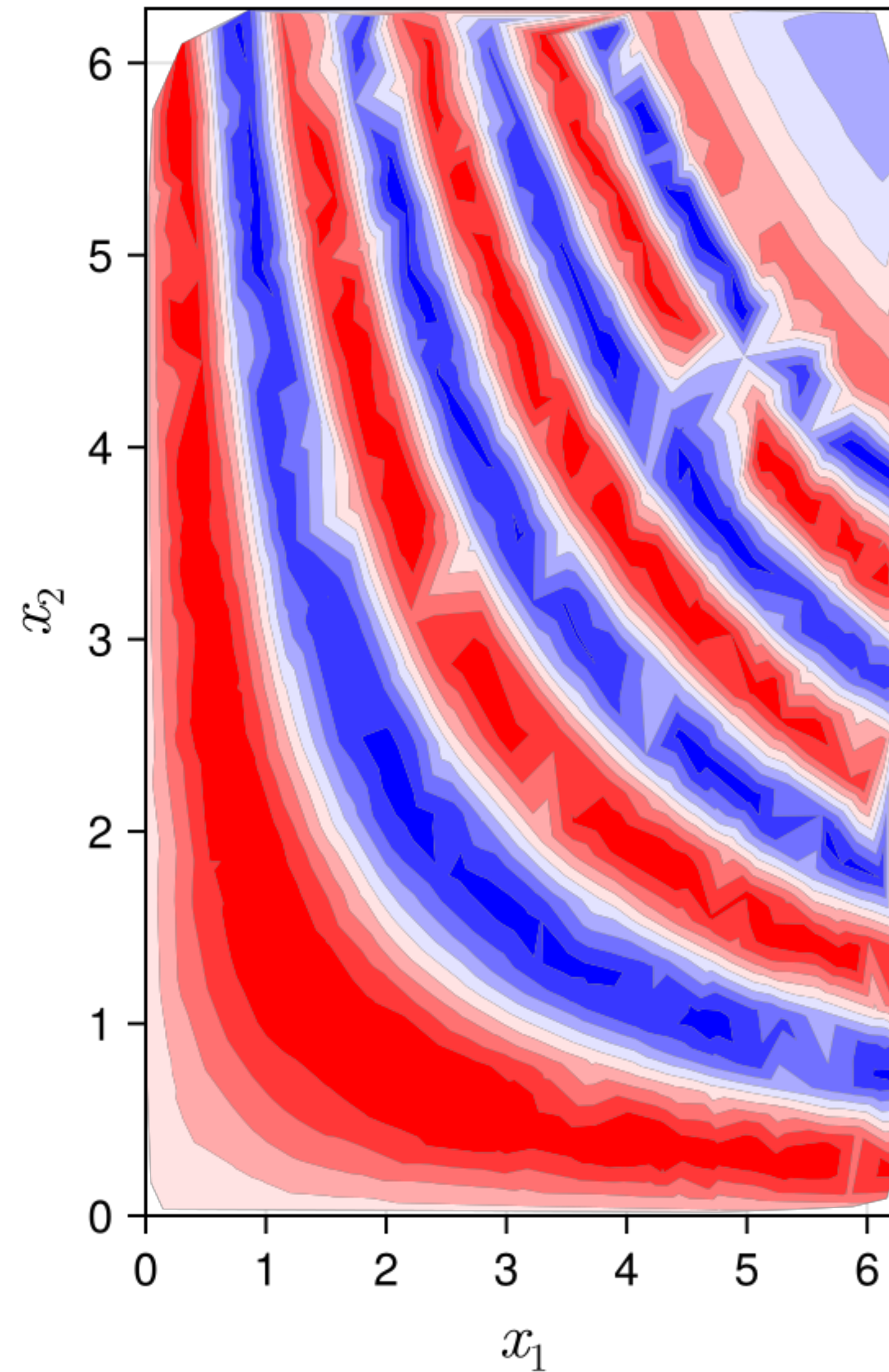
```
y_pred,_=model(x_samples,parameters, layer_states)
```



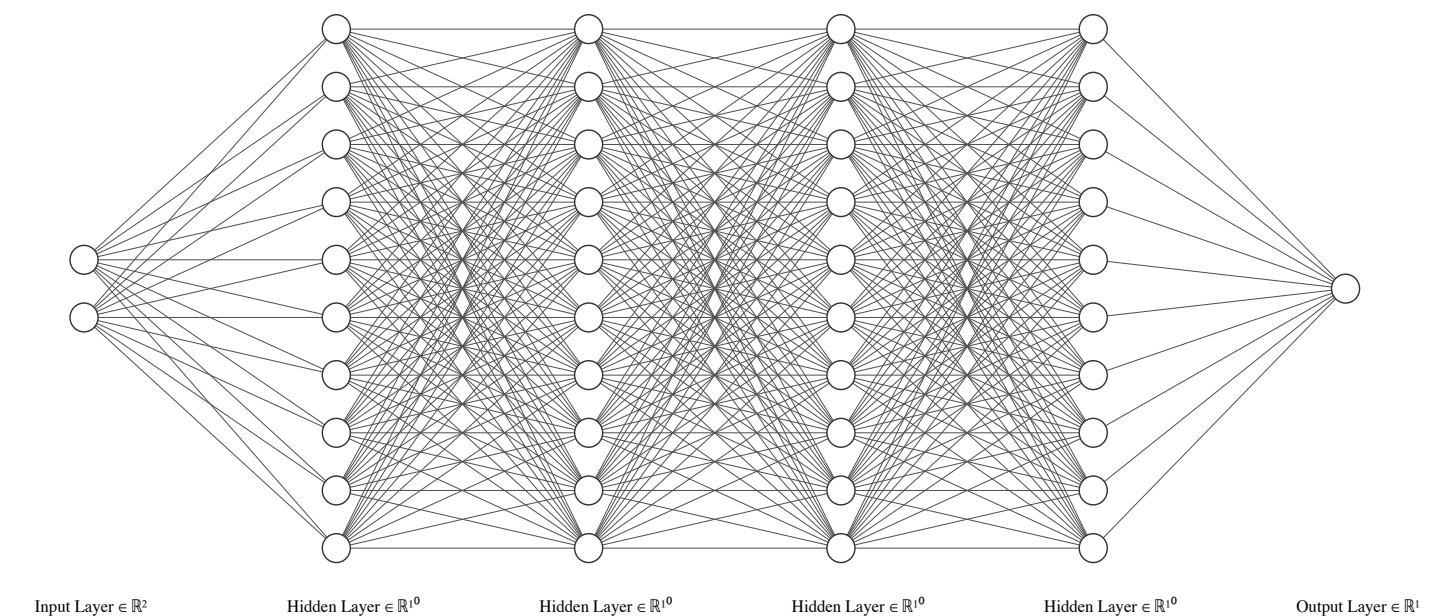
# Data



# Neural Model

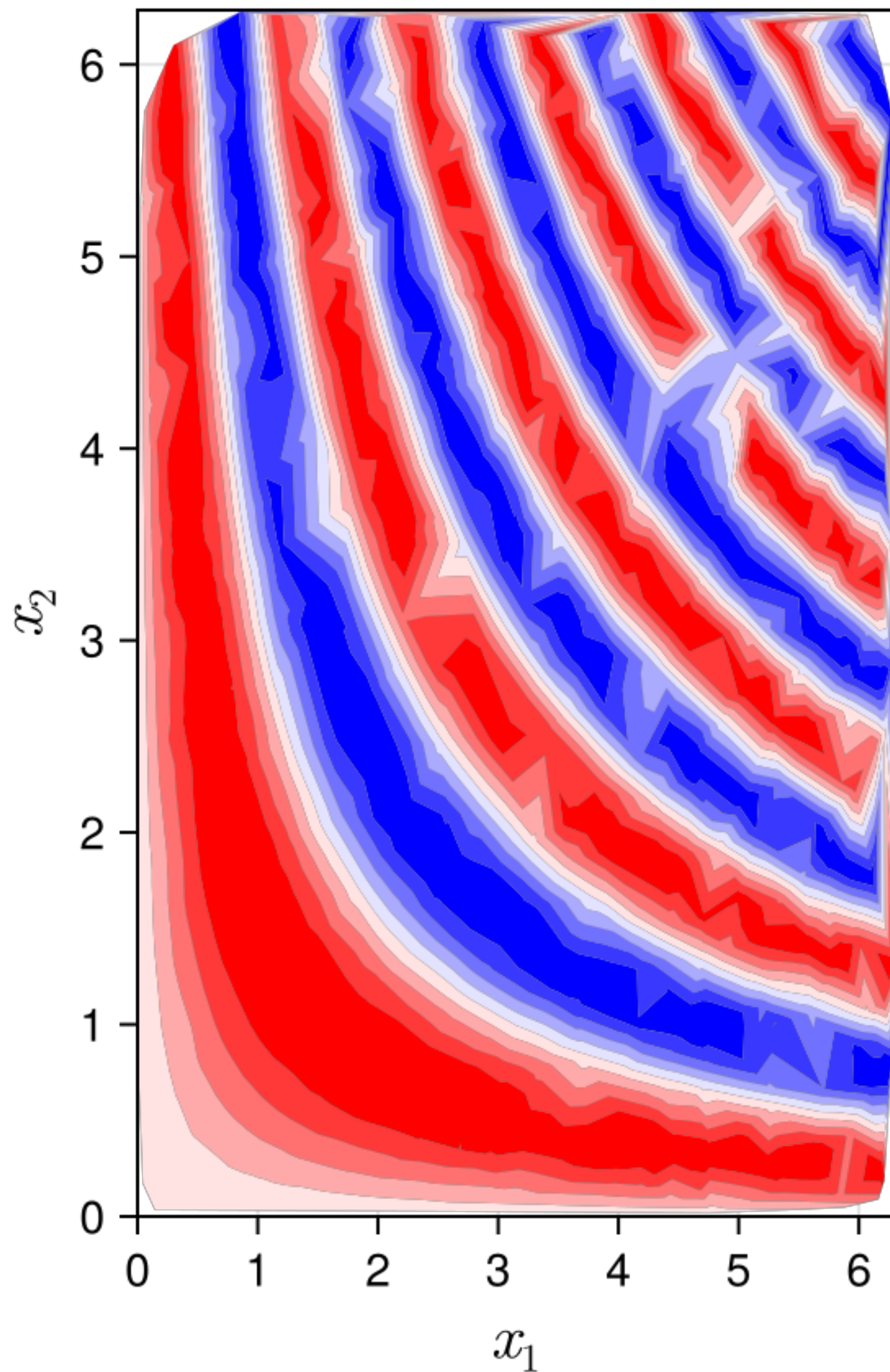


Agreement is  
much better!  
Pretty good model

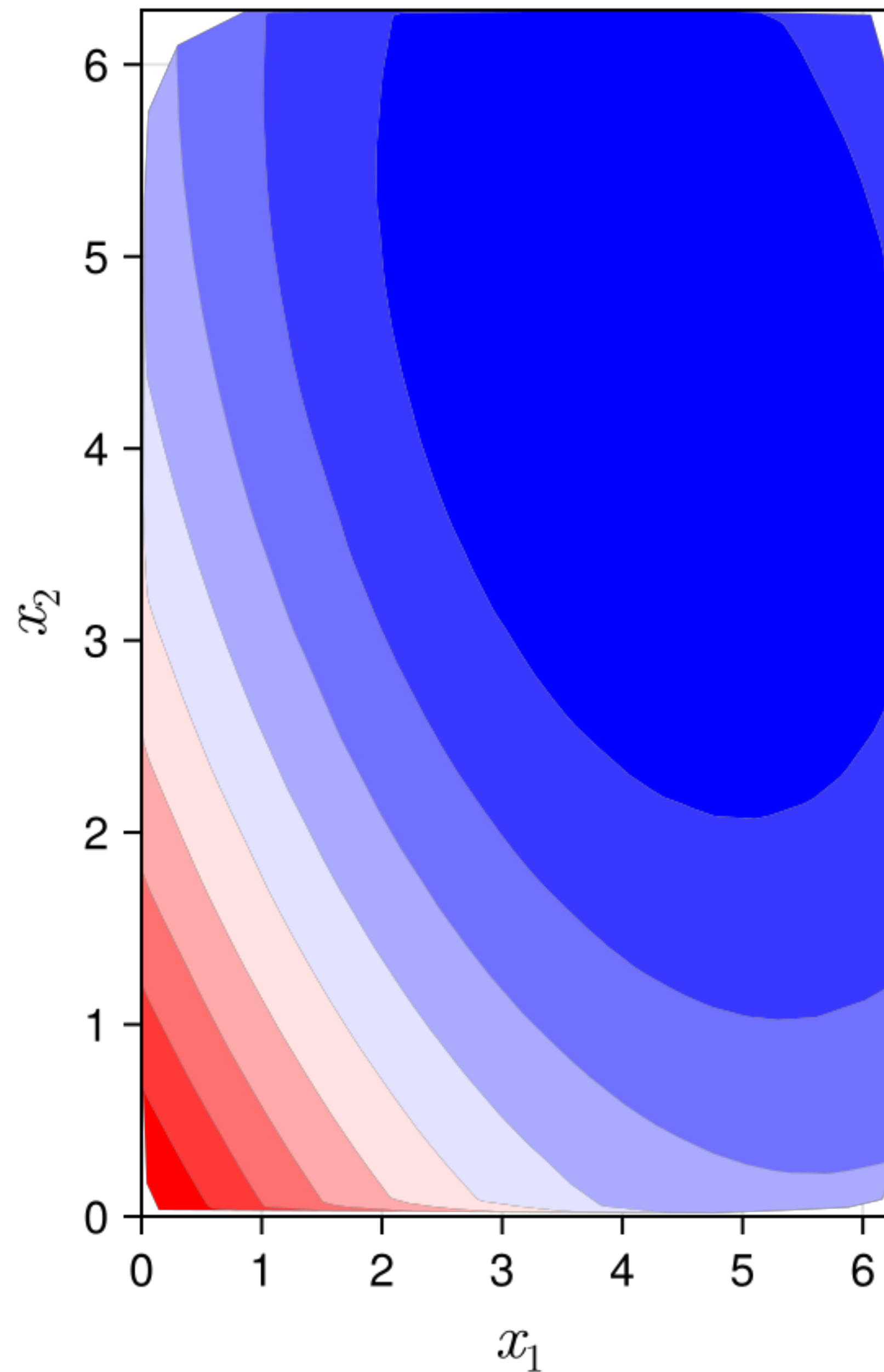




# Data



# Polynomial Model

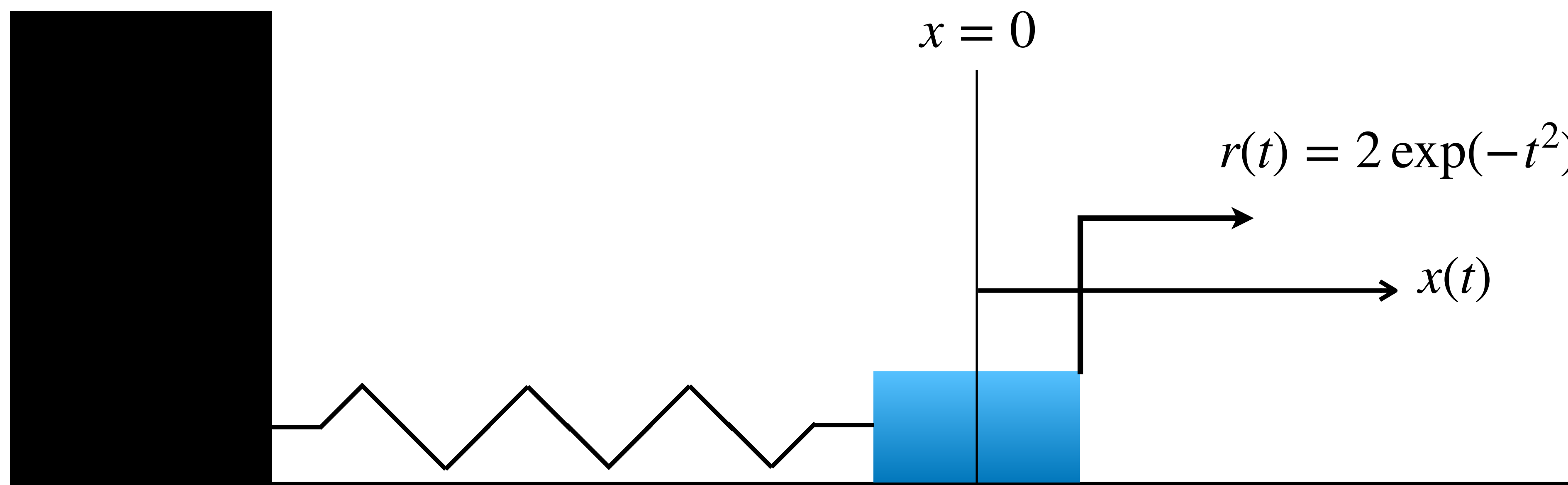


Agreement is  
not very good!  
Not a good model

$$f_{model}(x_1, x_2) = a_0 + a_1x_1 + a_2x_2 + a_3x_1x_2 + a_4x_1^2 + a_5x_2^2$$



# Scientific Data

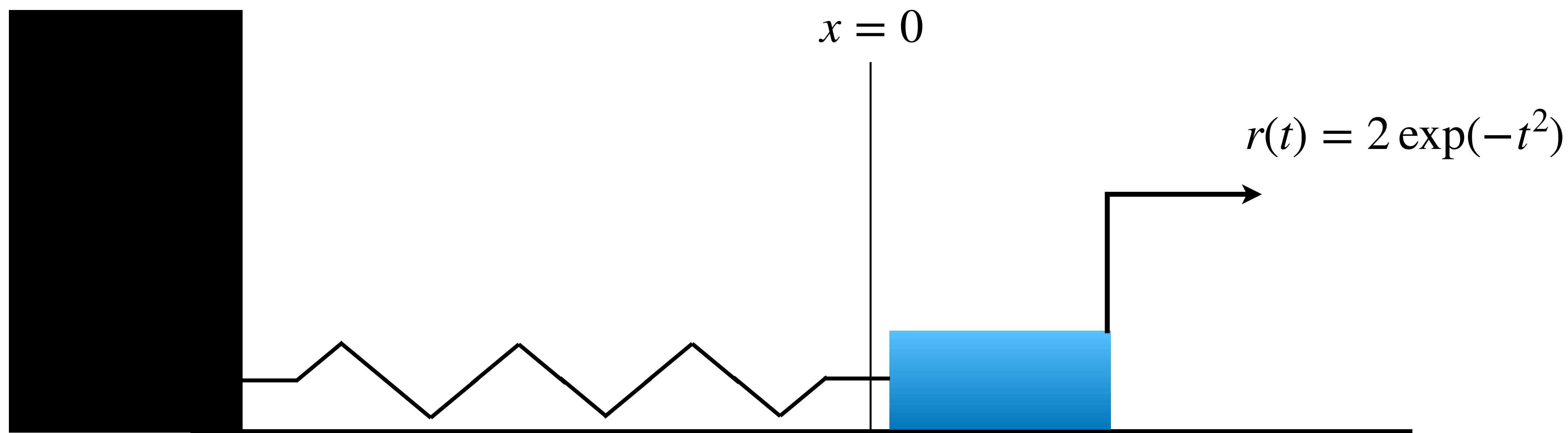


$$\frac{d^2x}{dt^2} + 4x = 2 \exp(-t^2)$$

$$x(0) = 0$$

$$\frac{dx}{dt}(t = 1) = 0$$

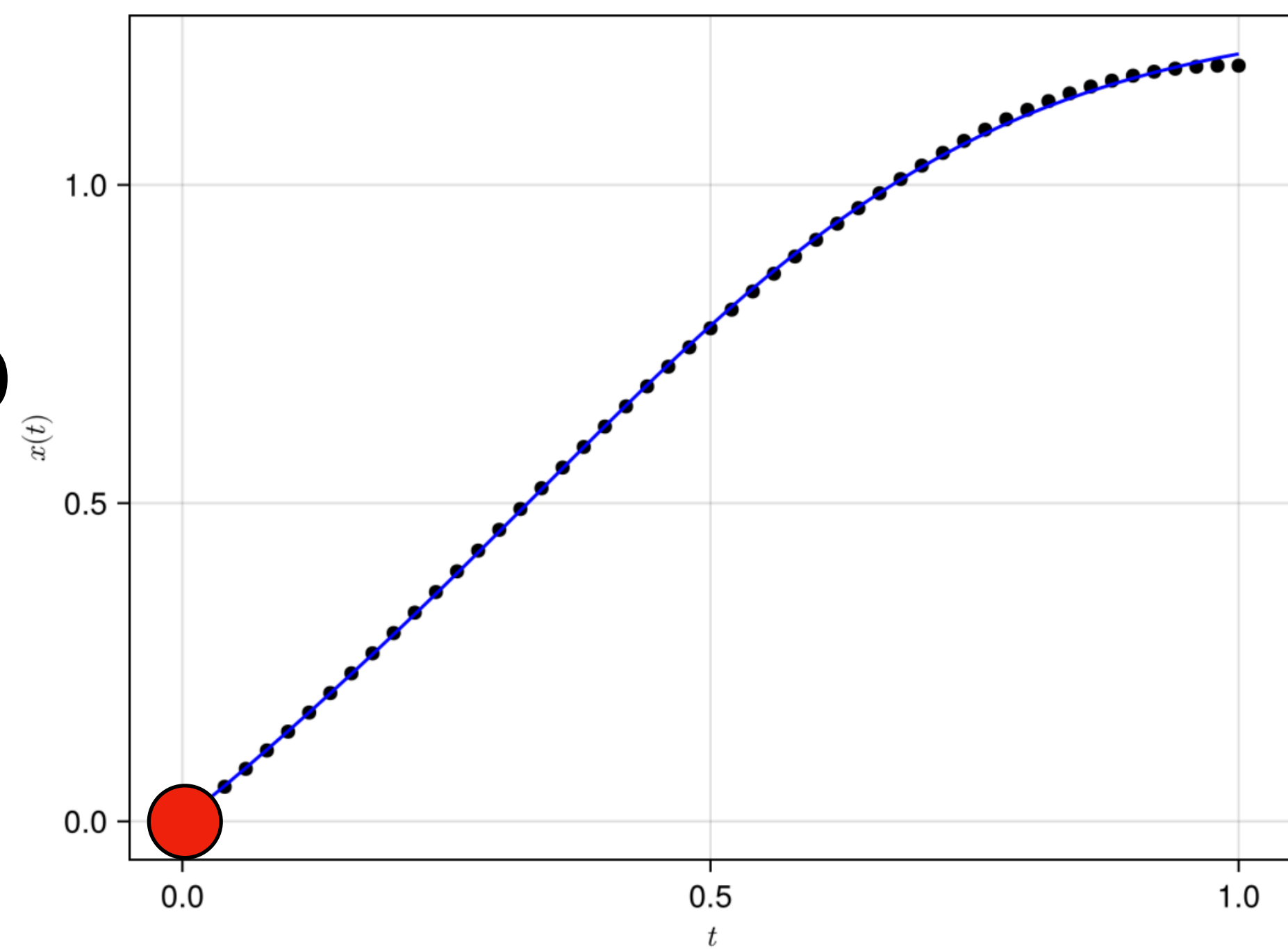


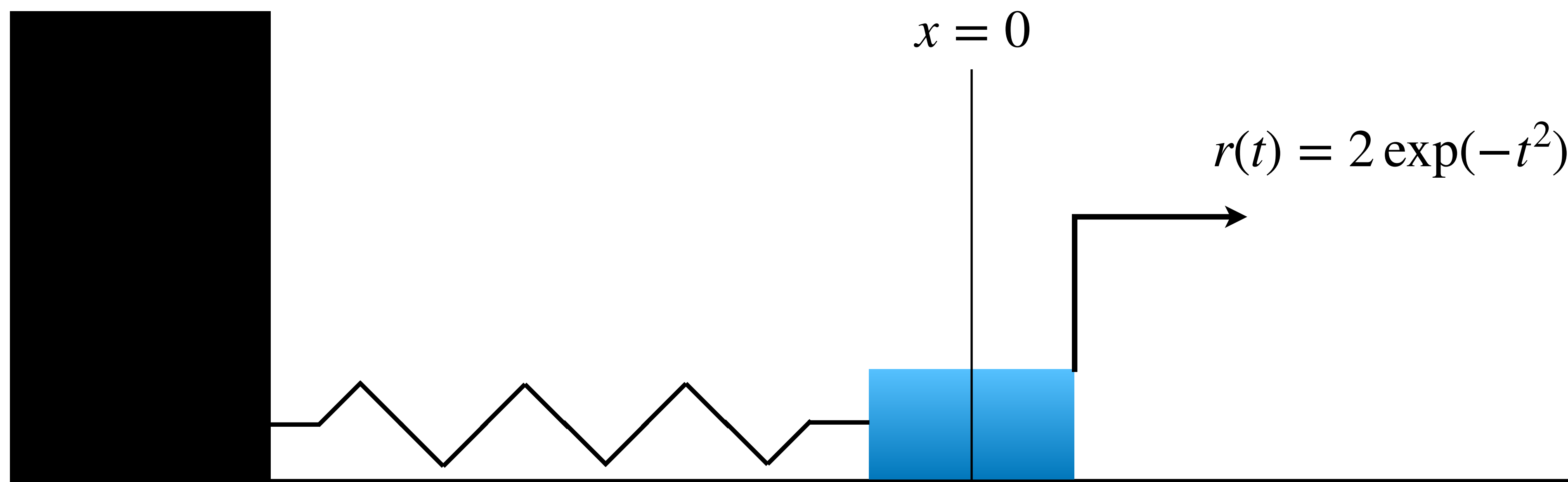


$$\frac{d^2x}{dt^2} + 4x = 2 \exp(-t^2)$$

$$x(0) = 0$$

$$\frac{dx}{dt}(t = 1) = 0$$



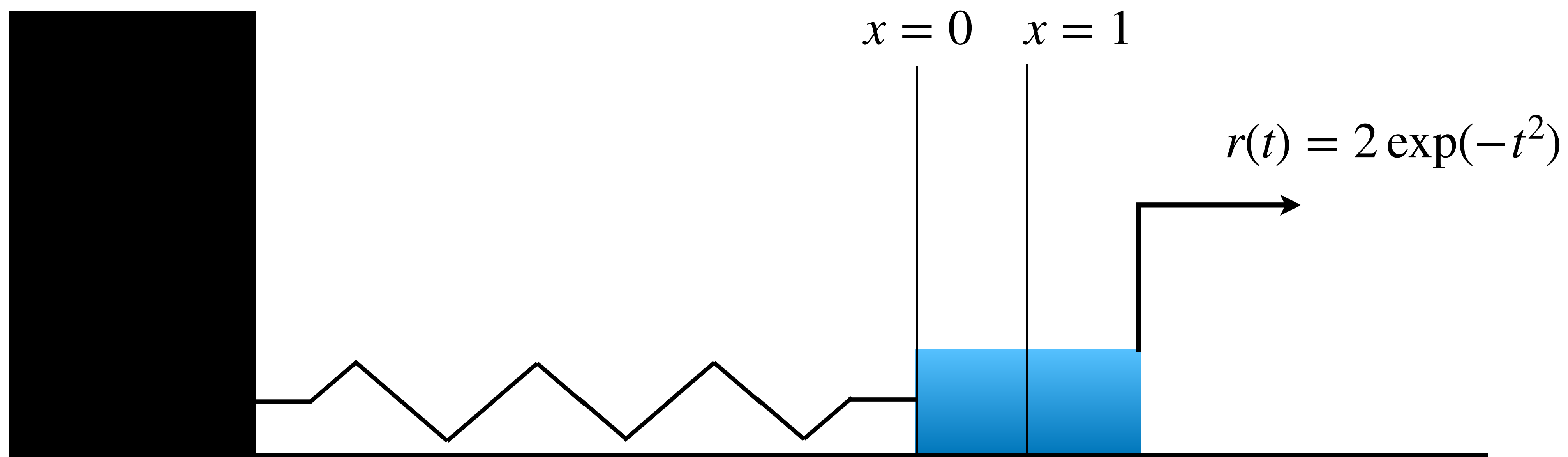


$$\frac{d^2x}{dt^2} + 4x = 2 \exp(-t^2)$$

$$x(0) = 0$$

$$\frac{dx}{dt}(t = 1) = 0$$

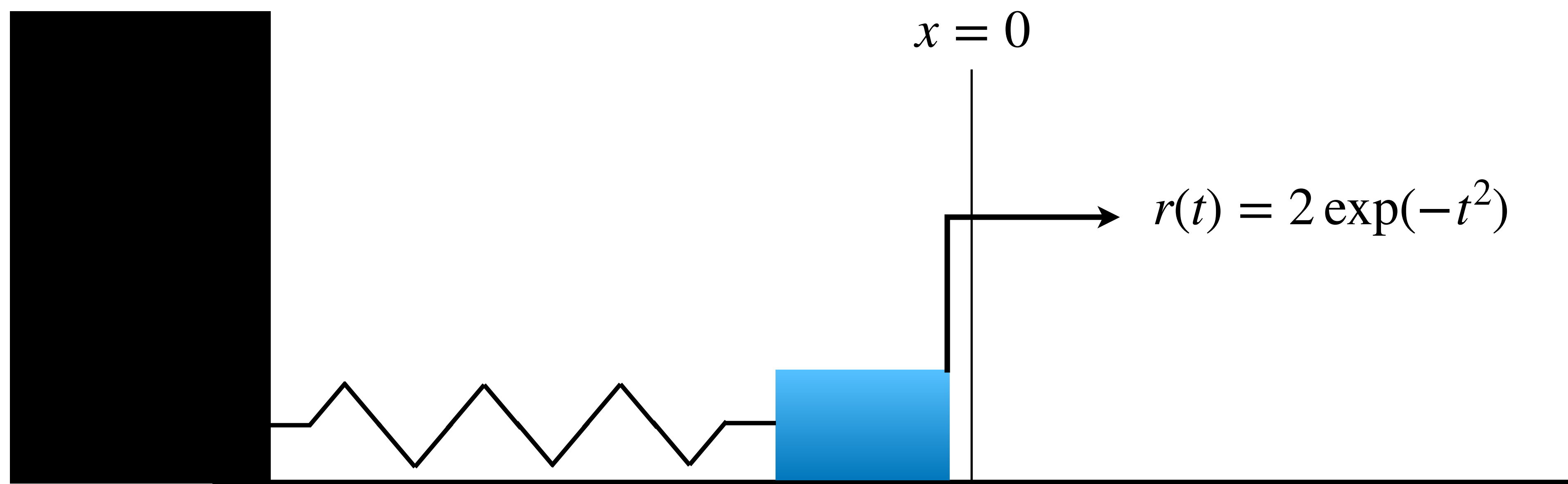




$$\frac{d^2x}{dt^2} + 4x = 2 \exp(-t^2)$$

$$x(0) = 1$$

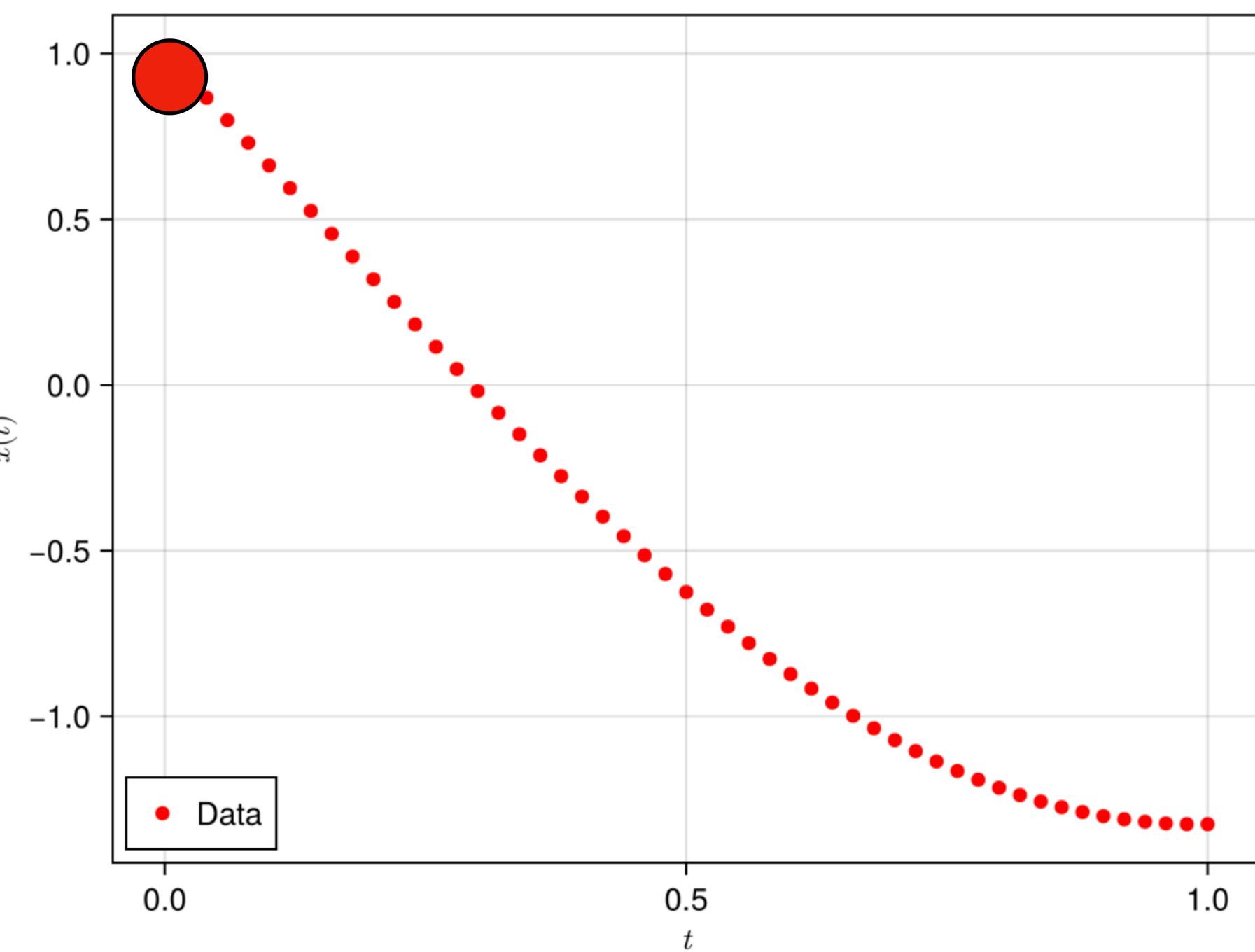
$$\frac{dx}{dt}(t = 1) = 0$$



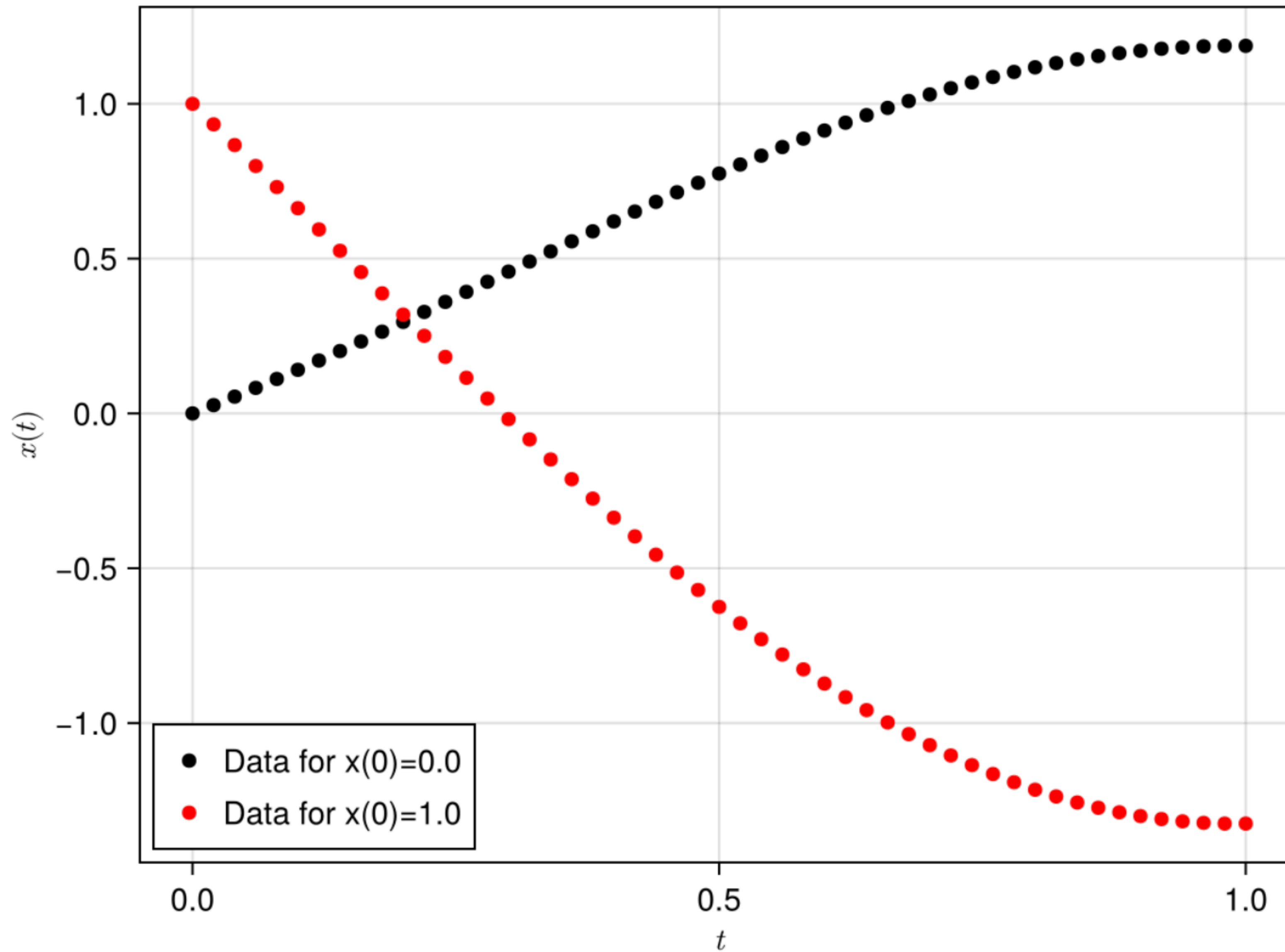
$$\frac{d^2x}{dt^2} + 4x = 2 \exp(-t^2)$$

$$x(0) = 1$$

$$\frac{dx}{dt}(t = 1) = 0$$







So we have two data sets

1. Initial location of the mass is at  $x(t=0) = 0$  (Data07.csv)

2. Initial location of the mass is at  $x(t=0) = 1$  (Data08.csv)

- Train the Neural Network model on Data07.csv and predict both data Data07.csv and Data08.csv.
- Train the Neural Network model on BOTH Data07.csv and Data08.csv and predict both data Data07.csv and Data08.csv.

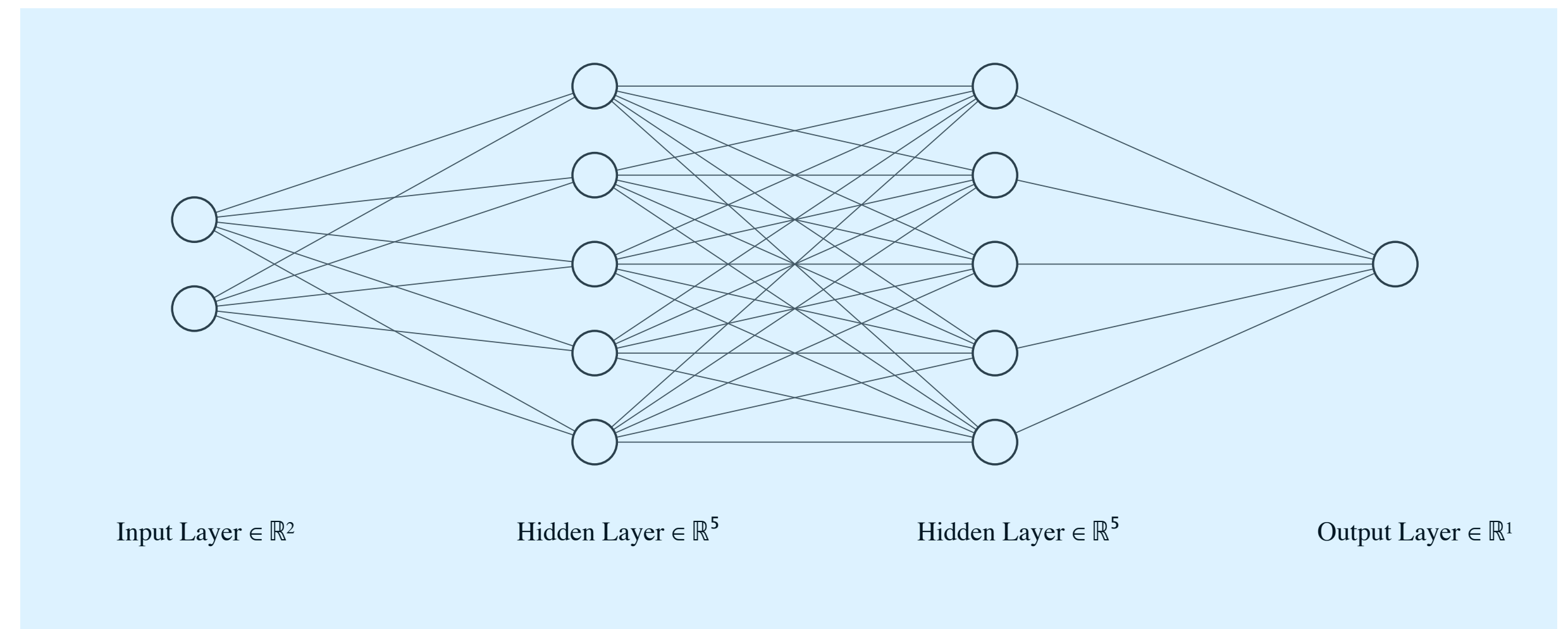
Read from data set, set up Neural Network and predict (no training)

```
df = CSV.read("Data07.csv", DataFrame)
tdata=df[:, :tdata]
xdata=df[:, :xdata]
xzero=df[:, :xzero]
```

Read data  
and assign to  
variables

$$\begin{bmatrix} 0 & 0 & 0 & \dots & 0 \\ t_1 & t_2 & t_3 & \dots & t_N \end{bmatrix}$$

```
t_input=[xzero'; tdata']
model = Chain(
    Dense(2 => 5),
    Dense(5 => 5, tanh),
    Dense(5 => 1)
)
```

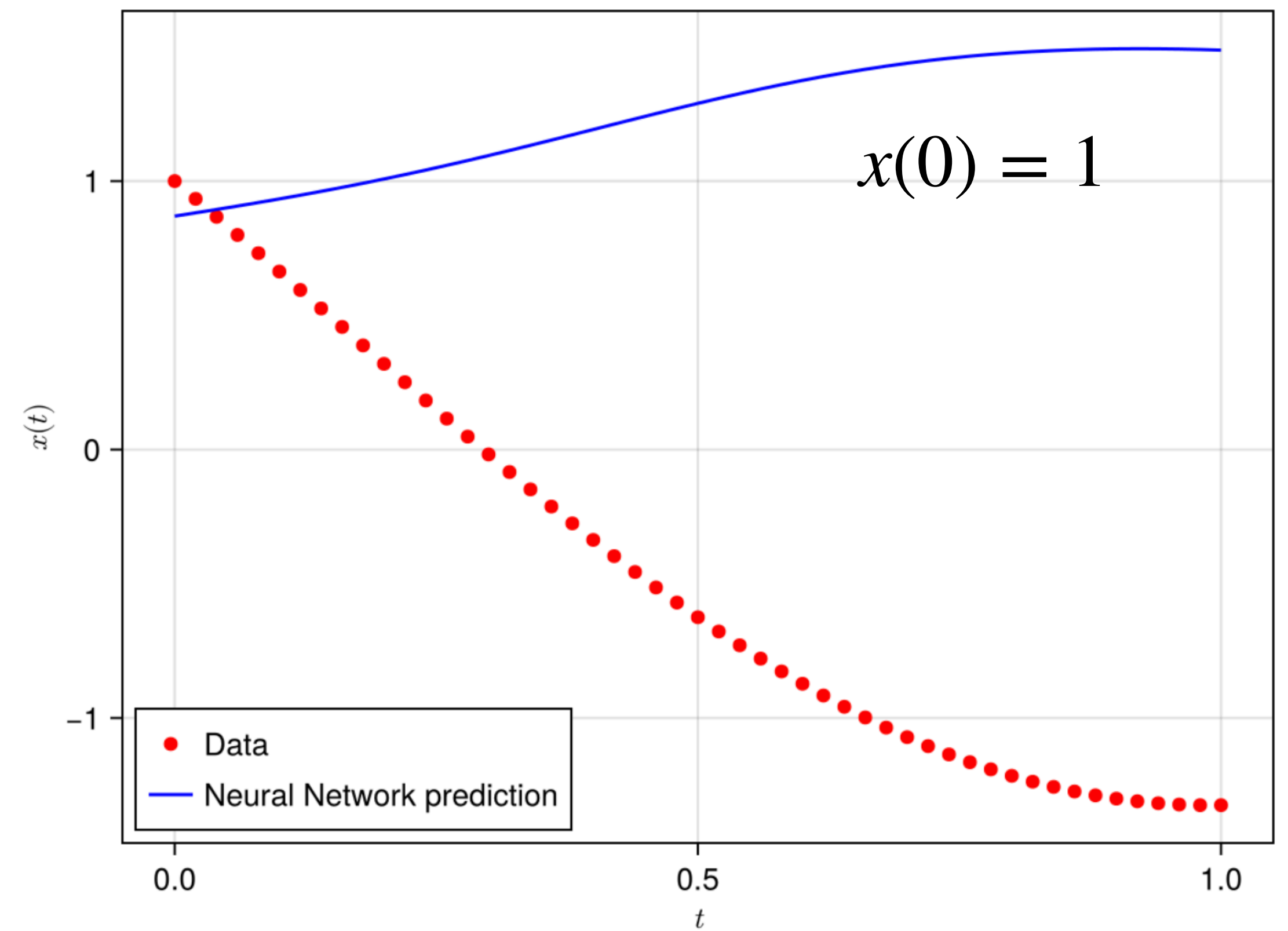
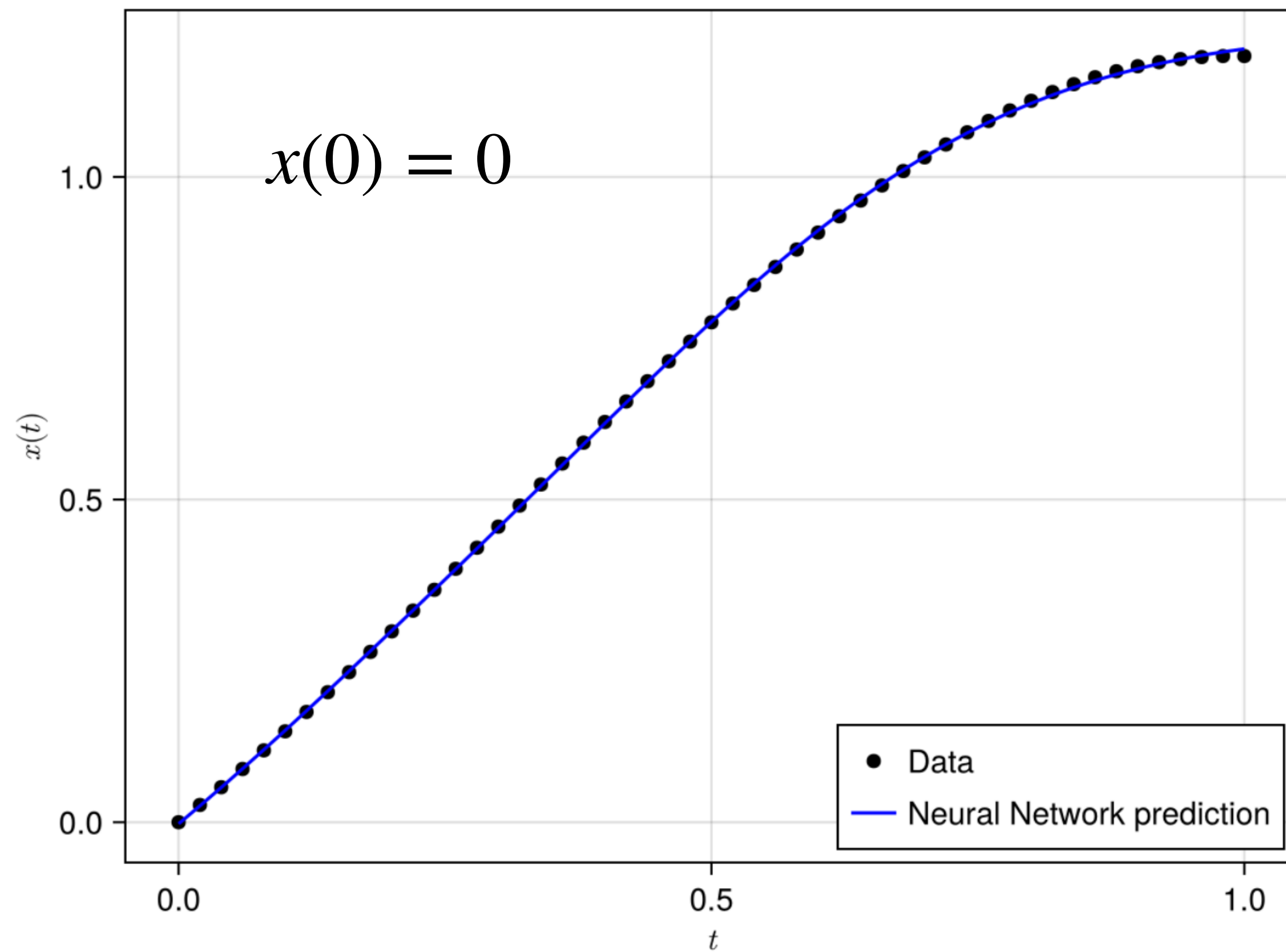


```
parameters, layer_states = Lux.setup(rng, model)
x_prediction, _ = model(t_input, parameters, layer_states)
```

$NN(\vec{x}, \mathbf{W}, \vec{b})$  Predict model using parameters



# Neural Network trained on $x(0) = 0$ data

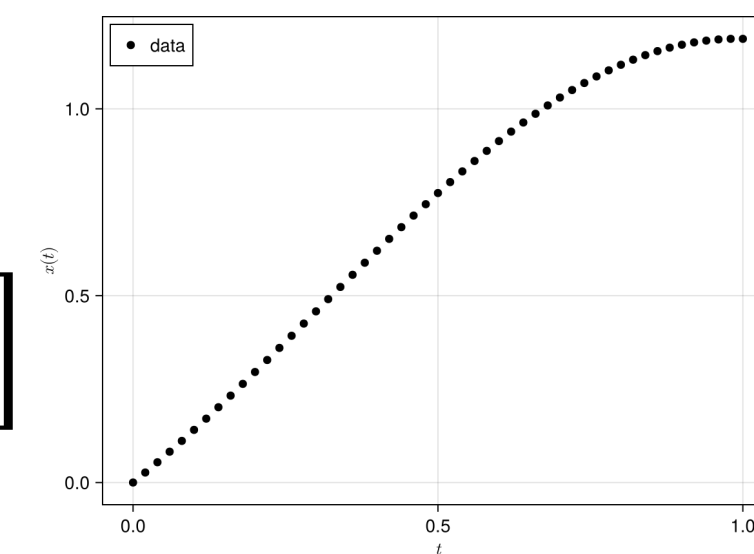


```
df1 = CSV.read("Data07.csv", DataFrame)
```

```
tdata1=df1[:, :tdata]  $[t_1 \ t_2 \ t_3 \ \dots \ t_N]$ 
```

```
xdata1=df1[:, :xdata]  $[x_1^{(1)} \ x_2^{(1)} \ x_3^{(1)} \ \dots \ x_N^{(1)}]$ 
```

```
xzero1=df1[:, :xzero]  $[0 \ 0 \ 0 \ \dots \ 0]$ 
```

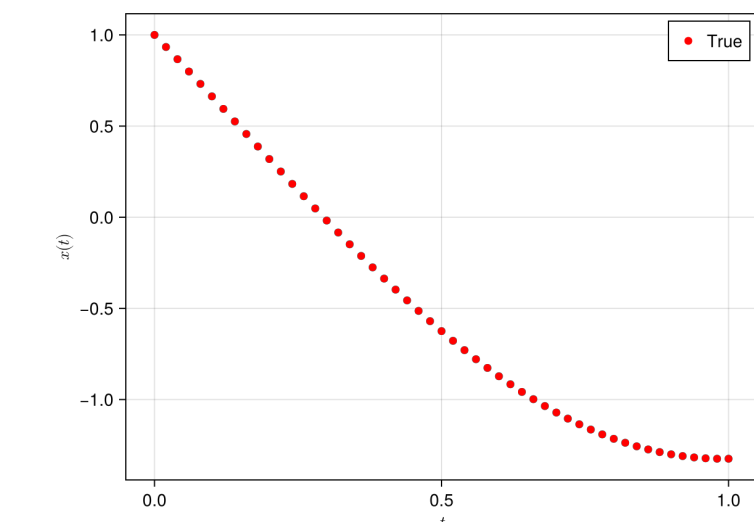


```
df2 = CSV.read("Data08.csv", DataFrame)
```

```
tdata2=df2[:, :tdata]  $[t_1 \ t_2 \ t_3 \ \dots \ t_N]$ 
```

```
xdata2=df2[:, :xdata]  $[x_1^{(2)} \ x_2^{(2)} \ x_3^{(2)} \ \dots \ x_N^{(2)}]$ 
```

```
xzero2=df2[:, :xzero]  $[1 \ 1 \ 1 \ \dots \ 1]$ 
```



```
t_input=vcat(hcat(xzero1', xzero2'),hcat(tdata1', tdata2'))
```

```
xdata_combined=hcat(xdata1', xdata2')
```

$$[x_1^{(1)} \ x_2^{(1)} \ x_3^{(1)} \ \dots \ x_N^{(1)} \ x_1^{(2)} \ x_2^{(2)} \ x_3^{(2)} \ \dots \ x_N^{(2)}]$$

```
model = Chain(
```

```
    Dense(2 => 5),
```

```
    Dense(5 => 5, tanh),
```

```
    Dense(5 => 1)
```

```
)
```

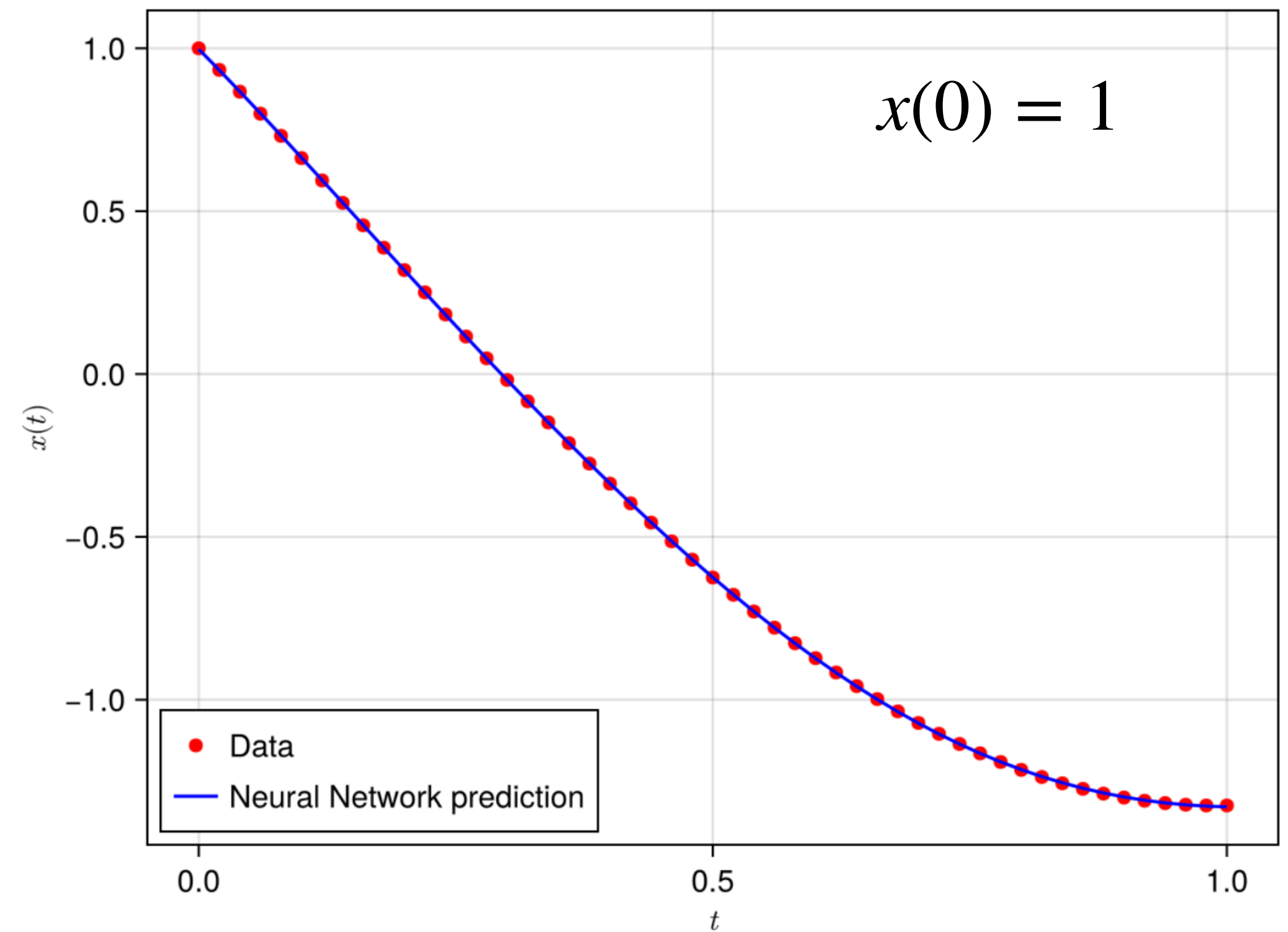
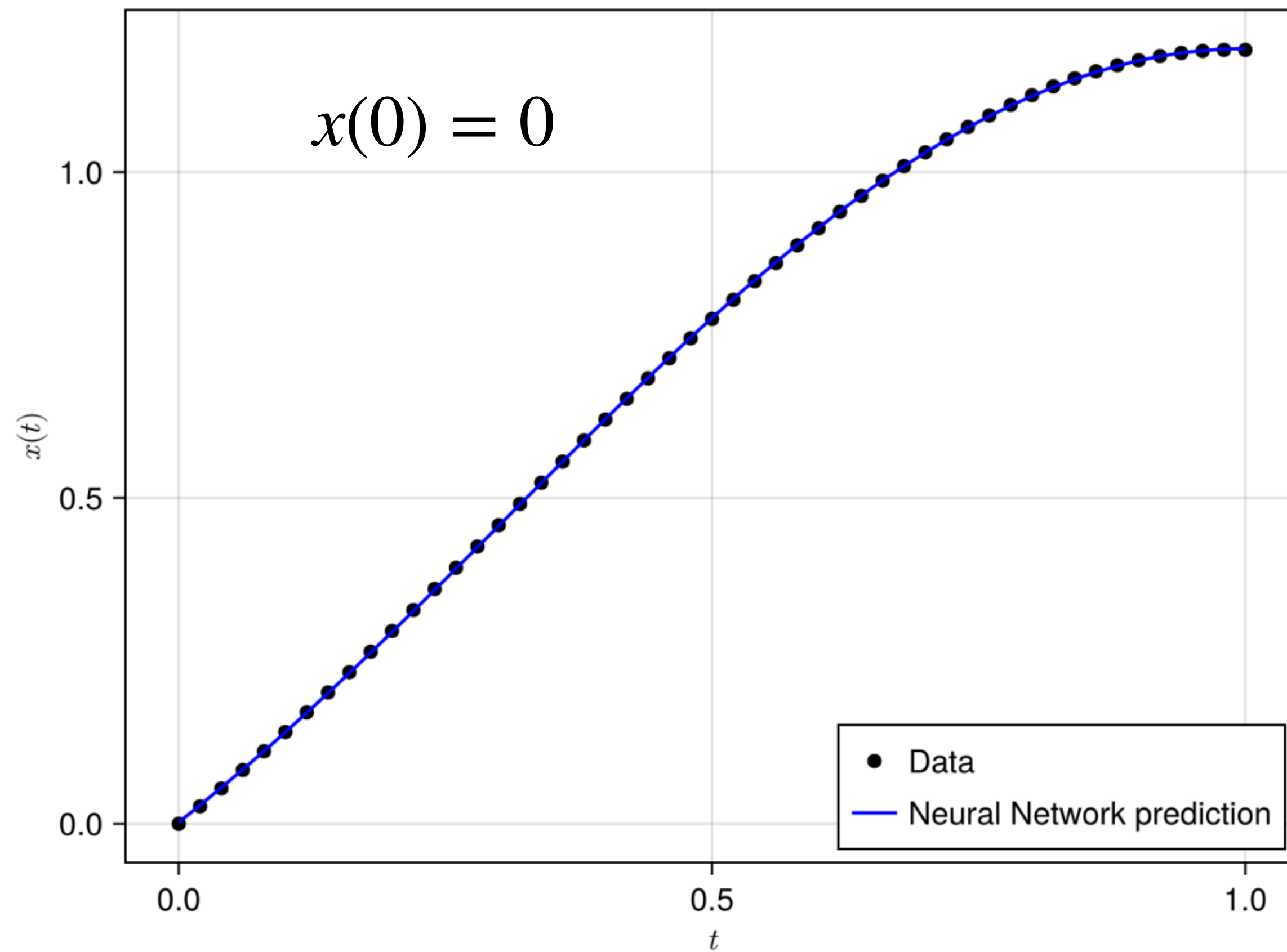
```
rng = Random.default_rng()
```

```
parameters, layer_states = Lux.setup(rng, model)
```

```
x_prediction, new_ls= model(t_input,parameters,layer_states)
```

$$\begin{bmatrix} 0 & 0 & 0 & \dots & 0 & 1 & 1 & 1 & \dots & 1 \\ t_1 & t_2 & t_3 & \dots & t_N & t_1 & t_2 & t_3 & \dots & t_N \end{bmatrix}$$

Neural Network trained on BOTH  $x(0) = 0$  data  
and  $x(0) = 1$





$$x(t = 0) = 0$$

Data07.csv

$$x(t = 0) = 1.0$$

Data08.csv

$$x(t = 0) = 2.0$$

Data09.csv

$$x(t = 0) = 0.5$$

Data10.csv